

Dicts

Hier steht `d` für ein Dict. Es behält die Reihenfolge, in der die Einträge angelegt wurden. Schlüssel müssen hashbar sein.

Werkzeug	Tut was	Gibt zurück
<code>d[key]</code>	liest den Wert zum Schlüssel	den Wert, bei fehlendem Schlüssel <code>KeyError</code>
<code>d.get(key, default)</code>	liest den Wert oder den Ersatzwert	den Wert oder <code>default</code> , ohne Angabe von <code>default</code> den Wert <code>None</code>
<code>key in d</code>	prüft die Schlüssel, nicht die Werte	<code>bool</code>
<code>d[key] = value</code>	legt den Eintrag an oder ersetzt den Wert	nichts, verändert <code>d</code>
<code>del d[key]</code>	entfernt den Eintrag	nichts, verändert <code>d</code> , bei fehlendem Schlüssel <code>KeyError</code>
<code>d.pop(key)</code>	entfernt den Eintrag	den Wert, verändert <code>d</code> , bei fehlendem Schlüssel <code>KeyError</code>
<code>len(d)</code>	Anzahl der Einträge	<code>int</code>
<code>d.keys()</code> , <code>d.values()</code> , <code>d.items()</code>	Schlüssel, Werte, Paare aus beiden zum Durchlaufen	eine Sicht auf das Dict
<code>d.update(other)</code>	übernimmt alle Einträge aus <code>other</code>	<code>None</code> , verändert <code>d</code>
<code>d.copy()</code>	flache Kopie	neues Dict

Tupel

```
hit = ("Gehirn", 0.89)           feste Reihenfolge, unveränderlich
hit[0], hit[-1]                 Zugriff wie bei einer Liste
title, score = hit              Auspacken: je Wert ein Name
a, b = b, a                     zwei Werte tauschen
single = ("Gehirn",)           ein Element braucht das Komma
for title, score in hits:       Auspacken in der Schleife
```

Tupel oder Liste? Trägt die Position die Bedeutung, passt das Tupel. Können Werte hinzukommen oder wegfallen, passt die Liste.

Muster mit Dicts und Listen

```
for key, value in d.items():      Einträge durchlaufen
counts[w] = counts.get(w, 0) + 1 zählen
by_id[p["id"]] = p              Nachschlagen vorbereiten
if title not in groups:         gruppieren: je Schlüssel
    groups[title] = []          eine Liste anlegen
groups[title].append(p["id"])    und darin sammeln
words.append(more_words)         ein Element: Liste in der Liste
words.extend(more_words)         jedes Element einzeln anhängen

append und extend verändern die Liste und geben None zurück.
```

Sets

```
s = set(words)                  jeder Wert einmal, ohne Reihenfolge
set()                           leeres Set, denn {} ist ein leeres Dict
x in s                           Prüfung, im Mittel unabhängig von der Größe
s.add(x)                         verändert s, Rückgabe None
a & b                             Schnittmenge: Werte in beiden Sets
a | b                             Vereinigung: Werte in mindestens einem
a - b                             Differenz: Werte in a, die in b fehlen
```

Set-Elemente und Dict-Schlüssel müssen hashbar sein. Zahlen, Strings und Tupel aus hashbaren Werten sind es, Listen, Dicts und Sets nicht.

Slicing

```
xs[2:5]                          Positionen 2, 3, 4: Start dabei, Stopp nicht
xs[:3]                           die ersten drei
xs[3:]                           ab Position 3 bis zum Ende
xs[-2:]                          die letzten zwei
xs[:-2]                          alles außer den letzten zwei
xs[::2]                          jedes zweite Element
xs[::-1]                          rückwärts
xs[5:100]                         über das Ende hinaus: kein Fehler
xs[100]                           einzelner Index über das Ende: IndexError
```

Slices gibt es auch auf Strings und Tupeln. `xs[:3]` und `xs[3:]` teilen eine Liste, ohne dass ein Element doppelt vorkommt oder fehlt.

Strings

Hier steht `s` für einen String. Keine der Methoden verändert `s`, denn Strings sind unveränderlich.

Werkzeug	Tut was	Gibt zurück
<code>s.lower()</code> , <code>s.upper()</code>	Klein- oder Großbuchstaben	neuer String
<code>s.strip()</code> , <code>s.strip(chars)</code>	entfernt Leerraum oder die Zeichen aus <code>chars</code> an beiden Enden	neuer String
<code>s.split()</code> , <code>s.split(sep)</code>	trennt an Leerraum oder am Trennzeichen <code>sep</code>	Liste von Strings
<code>sep.join(words)</code>	setzt eine Liste von Strings mit <code>sep</code> zusammen	neuer String
<code>s.replace(old, new)</code>	ersetzt jedes Vorkommen von <code>old</code>	neuer String
<code>s.startswith(x)</code> , <code>s.endswith(x)</code>	prüft Anfang oder Ende	<code>bool</code>
<code>x in s</code>	prüft, ob <code>x</code> als Teilstring vorkommt	<code>bool</code>
<code>s.find(x)</code>	sucht <code>x</code>	Position als <code>int</code> , <code>-1</code> wenn <code>x</code> fehlt
<code>s.count(x)</code>	zählt die Vorkommen von <code>x</code>	<code>int</code>
<code>len(s)</code>	Anzahl der Zeichen	<code>int</code>

Kopien

```
xs = [{"title": "Zeit"}, {"title": "Mali"}]
ys = xs[:]           flache Kopie
ys.append({"title": "Bern"})  xs hat weiter zwei Elemente
ys[0]["title"] = "Uhr"      auch xs[0]["title"] ist "Uhr"
```

Die flache Kopie ist eine neue Liste mit denselben Dicts. `xs.copy()` und `list(xs)` liefern dasselbe wie `xs[:]`. Anhängen, Entfernen und Ersetzen in `ys` lässt `xs` unverändert. Wird ein Dict selbst verändert, zeigt sich das in beiden Listen, denn es gibt jedes Dict nur einmal.

```
ys = [r.copy() for r in xs]      kopiert zusätzlich jedes Dict
ys = copy.deepcopy(xs)          kopiert alle Ebenen, Modul copy

r.copy() genügt, wenn die Werte der Dicts unveränderlich sind, also
Zahlen oder Strings. Stecken in den Dicts weitere Listen oder Dicts,
bleiben diese gemeinsam. Dann kopiert copy.deepcopy alle Ebenen.
```

Argumente an Funktionen

```
def add_flag(xs):
    xs.append("!")           verändert die übergebene Liste
def replace(xs):
    xs = ["neu"]             Zuweisung an den Parameter
data = [1, 2]
add_flag(data)              data ist jetzt [1, 2, "!"]
replace(data)               data bleibt [1, 2, "!"]
```

Python kopiert beim Aufruf das Objekt nicht. Der Parameter `xs` ist ein weiterer Name für das Objekt, das der Aufrufer übergibt (Call by Sharing). Verändert die Funktion dieses Objekt, sieht der Aufrufer die Änderung. Eine Zuweisung an den Parameter lässt nur den lokalen Namen `xs` auf ein anderes Objekt zeigen, die Liste des Aufrufers bleibt, wie sie ist.

Comprehensions

Eine Comprehension baut aus einer Liste eine neue Liste oder ein neues Dict. Die Beispiele verwenden diese Trefferliste:

```
hits = [("Gehirn", 0.89), ("Zeit", 0.72), ("Koran", 0.65)]
```

Comprehension	Ergebnis	Bedeutung
<code>[title for title, score in hits]</code>	<code>['Gehirn', 'Zeit', 'Koran']</code>	von jedem Treffer den Titel
<code>[title for title, score in hits if score >= 0.7]</code>	<code>['Gehirn', 'Zeit']</code>	nur von Treffern, für die die Bedingung gilt
<code>[title.upper() for title, score in hits]</code>	<code>['GEHIRN', 'ZEIT', 'KORAN']</code>	vorn steht ein Ausdruck, er darf rechnen
<code>{title: score for title, score in hits}</code>	<code>{'Gehirn': 0.89, 'Zeit': 0.72, 'Koran': 0.65}</code>	vorn ein Paar Schlüssel: Wert, außen geschweifte Klammern: Das Ergebnis ist ein Dict

```
good = [title for title, score in hits if score >= 0.7]
good = []
for title, score in hits:
    if score >= 0.7:
        good.append(title)
```

die zweite Zeile der Tabelle
dasselbe als Schleife

Aufbau: `[Ausdruck for Element in Liste if Bedingung]`. Sobald mehrere Schritte oder verschachtelte Schleifen nötig sind, ist die gewöhnliche Schleife leichter zu lesen und zu prüfen.

Eingebaute Funktionen

Hier steht `xs` für eine Liste, ein Tupel oder einen anderen Container. Keine dieser Funktionen verändert ihre Eingabe.

Werkzeug	Tut was	Gibt zurück
<code>len(xs)</code>	Anzahl der Elemente einer Liste, eines Tupels, Strings, Dicts oder Sets	<code>int</code>
<code>reversed(xs)</code>	durchläuft eine Liste, ein Tupel oder einen String rückwärts	etwas zum Durchlaufen, mit <code>list(...)</code> eine Liste
<code>min(xs), max(xs)</code>	kleinstes und größtes Element, auch mit <code>key=f</code>	ein Element
<code>sum(xs)</code>	Summe, die Elemente sind Zahlen	eine Zahl
<code>any(xs)</code>	prüft, ob mindestens ein Element wahr ist	<code>bool</code>
<code>all(xs)</code>	prüft, ob alle Elemente wahr sind	<code>bool</code>

`min`, `max`, `sum`, `any` und `all` arbeiten wie `sorted`, `enumerate` und `zip` auf allem, was sich mit `for` durchlaufen lässt. `len` braucht einen Container, dessen Anzahl feststeht. Zwei Beispiele: `max(hits, key=lambda h: h[1])` liefert den Treffer mit dem höchsten Score. `any([s > 0.8 for t, s in hits])` prüft, ob mindestens ein Score über 0,8 liegt.

Leere Eingabe: `sum([])` ergibt 0, `any([])` ergibt False und `all([])` ergibt True. `min([])` und `max([])` brechen mit einem `ValueError` ab.

enumerate und zip

```
enumerate(hits, start=1)      Paare aus Nummer und Element
for rank, (title, score) in enumerate(hits, start=1):
    zip(titles, scores)       Paare von derselben Position
for title, score in zip(titles, scores):
    list(zip(titles, scores))  Liste von Tupeln
zip(titles, scores, strict=True)  ungleich viele: ValueError
```

Ohne `strict=True` endet `zip` mit der kürzeren Liste, ohne Fehlermeldung.

Sortieren

```
sorted(xs)                   neue Liste, aufsteigend
sorted(xs, reverse=True)     absteigend
sorted(words, key=len)       nach der Anzahl der Zeichen
sorted(words, key=str.lower) Groß und Klein gleich behandelt
sorted(hits, key=lambda h: h[1]) nach dem Score an Position 1
xs.sort()                    ordnet xs selbst, Rückgabe None
```

- `key` bekommt eine Funktion ohne Klammern: `key=len`, nicht `key=len()`. `lambda h: h[1]` ist eine Funktion ohne Namen.
- Bei gleichem Schlüssel bleibt die Reihenfolge der Eingabe erhalten, auch mit `reverse=True`.
- Strings werden Zeichen für Zeichen nach Unicode-Zahlenwerten verglichen: A bis Z vor a bis z, die deutschen Umlaute dahinter. Das ist keine deutsche alphabetische Ordnung.
- Nie `xs = xs.sort()`: Danach ist `xs` keine Liste mehr.
- Die k besten Treffer: `sorted(hits, key=lambda h: h[1], reverse=True)[:k]`